

Capstone Project Notes: RAGBench, RGB, Proposal, Evaluation, and Timeline

1. Immediate Action Items

Proposal Document

- The proposal document is due **today or tomorrow**.
- **Tomorrow is the deadline.**
- Teams should internally discuss, finalize, and submit the proposal.
- Tomorrow, each team must give a **walkthrough of the proposal document** to their respective mentor.
- A PPT is **not required** for the proposal walkthrough. Teams can directly walk through the proposal document.

Mentors

- There are around **14-15 groups**.
 - Gopi will handle around **7 groups**.
 - Logesh will handle the remaining groups.
 - Mentor allocation details will be shared by the end of the day.
 - From next week onward, breakout rooms will be created for each group.
 - Each team will work in its own breakout room, and mentors will join to answer questions.
-

2. Project Overview

The project is about building, evaluating, and optimizing RAG systems.

During the earlier hackathon, teams likely built a basic RAG application with the following pipeline:

1. Read documents using a document parser.
2. Convert documents into text.
3. Break text into chunks.
4. Store chunks in a vector database using an embedding model.
5. For a user query, perform similarity search.
6. Retrieve top-k relevant chunks.
7. Pass the query and retrieved chunks to an LLM.
8. Generate the final response.

This capstone project goes deeper. The goal is to optimize each major component of the RAG pipeline depending on the domain.

The main components to optimize are:

- Chunking
- Embedding
- Vector database

- Retrieval
 - Re-ranking
 - Context presentation
 - Summarization
 - Evaluation
-

3. RAG Pipeline Components to Experiment With

3.1 Chunking

Chunking is the first major component after documents are collected.

Different chunking strategies can be tested, such as:

- Fixed-size / pre-defined chunking
- Sliding-window-based chunking
- Semantic chunking
- Domain-specific chunking
- Chunking with metadata

Important point:

A chunking strategy that works well for one domain may not work well for another.

For example:

- Legal data may require a different chunking approach.
- Financial data may require another.
- Medical or biomedical data may need yet another.

Teams are expected to experiment with different chunking strategies and analyze which strategy works best for each domain.

3.2 Embedding Models

The embedding model is another key component.

An embedding model that works well for legal data may not work well for financial, medical, or customer-support data.

Teams should not simply use a generic embedding model for every domain.

Embedding models should be selected based on:

- Domain suitability
- Downstream task performance
- Memory constraints
- Retrieval quality

A suggested place to explore embedding models is the **MTEB leaderboard**.

MTEB stands for **Massive Text Embedding Benchmark**.

It contains many embedding models ranked across benchmarks. Teams can select models from MTEB based on performance and available compute/memory.

3.3 Vector Databases

Different vector databases can also be tested.

Examples mentioned:

- Chroma
- FAISS, which stands for Facebook AI Similarity Search
- Milvus
- Qdrant
- Weaviate

Teams can start with simpler options like:

- Chroma
- FAISS

Then they can experiment with enterprise-grade options like:

- Milvus

Expected differences between vector databases may include:

- Latency
- Metadata support
- Storage capabilities
- Retrieval performance

However, vector DB experimentation may not be as extensive as chunking, embedding, and retrieval. Trying at least a couple of vector databases is encouraged.

3.4 Retrieval

Retrieval is considered the heart of the RAG application.

Different retrieval techniques include:

Sparse Retrieval

Sparse retrieval uses techniques such as:

- BM25

Dense Retrieval

Dense retrieval uses dense embedding models or LLM-based embeddings to create vector representations.

Hybrid Search

Hybrid search combines:

- Sparse retrieval
- Dense retrieval

This can improve retrieval quality by using both keyword-based and semantic similarity-based retrieval.

3.5 HyDE: Hypothetical Document Embedding

HyDE stands for **Hypothetical Document Embedding**.

Instead of querying the vector database directly with the question, the system first generates a hypothetical answer or document for the question.

Example:

Question:

Who won the IPL in 2018?

Instead of using this exact question to search the vector database, the system first generates a hypothetical answer and then uses that generated answer/document to retrieve relevant documents.

HyDE can also be combined with hybrid search.

3.6 Query Rewriting and Query Decomposition

Teams can also improve retrieval by applying:

- Query rewriting
- Query decomposition

These techniques can help improve retrieval quality, especially for complex or multi-part questions.

3.7 Re-ranking

After retrieving top-k documents, teams can re-rank them based on importance.

Re-ranking controls how the retrieved context is presented to the LLM.

Possible techniques include:

- MonoT5
- Cross-encoders
- Other re-ranking techniques

Teams should experiment with whether more important chunks should be placed:

- At the beginning of the prompt
- At the end of the prompt
- In another order

Different context-placement strategies such as forward and reverse ordering can be explored.

3.8 Summarization

Instead of passing all retrieved top-k documents directly to the LLM, teams can summarize the retrieved context.

Example:

If five documents are retrieved, the system can summarize them into a concise paragraph before sending them to the LLM.

Summarization techniques include:

- Extractive summarization
- Abstractive summarization

This may help reduce prompt size and improve response quality.

4. Main Benchmarks

The project uses two main benchmarks:

1. RAGBench
2. RGB

These are two separate tasks.

Part 1: RAGBench

5. RAGBench Overview

RAGBench is a large-scale benchmark dataset for evaluating RAG systems.

It contains around **100,000 data points** across five industry-specific domains.

The five domains are:

1. Biomedical research
2. General knowledge
3. Legal
4. Customer support
5. Finance

The dataset is available through a Hugging Face dataset link shared in the project materials.

Teams will build domain-specific RAG architectures and try to improve evaluation metrics for each domain.

6. RAGBench Evaluation Metrics

The key RAGBench evaluation metrics are:

1. Context relevance
2. Context utilization
3. Completeness
4. Adherence

6.1 Context Relevance

Context relevance measures how many of the retrieved documents or chunks are actually relevant to the query.

In simple terms:

Out of the documents retrieved for a query, how many are useful and relevant?

6.2 Context Utilization

Context utilization measures how much of the retrieved context is actually used by the LLM to generate the response.

In simple terms:

Out of the retrieved context, how much did the LLM actually use?

6.3 Completeness

Completeness measures whether the generated response includes all the relevant information from the context.

Example:

If there are three relevant sentences in the retrieved context, completeness checks whether the LLM's answer incorporates the information from all three.

In simple terms:

Did the response cover all necessary relevant information?

6.4 Adherence

Adherence is related to hallucination checking.

It measures whether all parts of the generated response are grounded in the provided context.

In simple terms:

Did the LLM only say things that are supported by the context?

The system should avoid generating unsupported information.

7. RAGBench Objective

For each domain, teams should build and optimize a RAG system.

The goal is to improve:

- Context relevance
- Context utilization
- Completeness
- Adherence

Teams should analyze what changes in the RAG pipeline improve these metrics.

For example:

- Does changing the chunking strategy improve context relevance?
 - Does changing the embedding model improve retrieval?
 - Does re-ranking improve completeness?
 - Does summarization improve adherence?
-

8. RAGBench Dataset Usage

Teams will work with JSON/text data, not original PDFs.

Important clarification:

- The project does not require using original PDF documents.
- The benchmark dataset already contains extracted text.
- There is no available mapping back to the original PDFs.
- Teams should work with the provided text/document fields.

For each question in RAGBench:

- There is a list of relevant documents.
 - Teams should take those documents.
 - Apply chunking.
 - Store the chunks in a vector database.
 - Run retrieval and response generation.
 - Evaluate using the given metrics.
-

9. RAGBench Evaluation Implementation

The evaluation metrics must be implemented by the teams.

There is no ready-made library expected to be used for these metrics.

Teams should refer to the RAGBench paper, where:

- Metric definitions are provided.
- Formulas are provided.
- Evaluation methodology is explained.

Important clarification:

The project is not simply about comparing the generated answer to the reference answer.

Instead, teams must compute the RAGBench metrics:

- Context relevance
- Context utilization
- Completeness
- Adherence

Evaluation is a key part of the project, not an optional add-on.

10. Reference Scores and Comparison

The dataset contains reference scores.

Teams should compare their computed scores against the reference scores in the dataset.

Example:

If the dataset has a context relevance score of 0.5 for a particular question, the team's experiment should try to produce a score closer to that reference.

The objective is not necessarily to get the exact same score.

The objective is to:

- Build the RAG pipeline.
 - Implement evaluators.
 - Run experiments.
 - Compare against reference scores.
 - Analyze why scores are high or low.
 - Optimize the pipeline accordingly.
-

11. Experiment Design

Teams should experiment with combinations of pipeline components.

Example combination:

- A = Chunking strategy
- B = Embedding model
- C = Retrieval mechanism
- D = LLM

A complete experiment may be represented as:

$A + B + C + D$

Then teams can change one component and observe the impact.

Example:

Original combination:

$A + B + C + D$

Change chunking strategy from A to X:

$X + B + C + D$

Then compare whether metrics improved or declined.

This should be repeated across domains and components.

12. Expected Final Outcome for RAGBench

The final output should include:

- A matrix of strategies and results.
- Which strategy worked best for each domain.
- Which chunking approach worked best for each domain.
- Which embedding model worked best for each domain.
- Which vector database/retrieval method worked best.
- Which re-ranking or summarization approach helped.
- Reasoning behind why certain combinations worked better.
- Analysis of why some combinations performed poorly.
- Recommendations for improving domain-specific RAG.

The project should include not only experiments and numbers, but also thoughtful analysis.

Part 2: RGB

13. RGB Overview

RGB is a separate benchmark from RAGBench.

RGB evaluates the **generation capability of LLMs**.

Unlike RAGBench, RGB is not focused on evaluating:

- Retriever
- Embedder
- Vector database

Instead, RGB focuses on whether the LLM can generate correct responses under different challenging conditions.

RGB evaluates four abilities:

1. Noise robustness
 2. Negative rejection
 3. Information integration
 4. Counterfactual robustness
-

14. RGB Ability 1: Noise Robustness

Noise robustness checks whether the LLM can answer correctly when noisy but related documents are present.

Example question:

Who was awarded the 2022 Nobel Prize in Literature?

There may be two documents:

- Document 1 contains the correct answer for 2022.
- Document 2 is related to Nobel Prize in Literature but talks about 2021.

The second document is relevant to the topic but does not contain the answer.

The evaluation checks whether the LLM:

- Picks the answer from the correct document.
 - Ignores noisy related documents.
 - Avoids being misled by irrelevant or partially relevant information.
-

15. RGB Ability 2: Negative Rejection

Negative rejection checks whether the LLM refuses to answer when the provided documents do not contain the answer.

Example question:

Who was awarded the 2022 Nobel Prize in Literature?

Provided documents may discuss:

- 2021 Nobel Prize in Literature
- 2020 Nobel Laureate in Literature

None of the documents contain the correct 2022 answer.

In this case, the ideal LLM response should be something like:

I do not have sufficient information to answer this question.

The evaluation checks whether the LLM:

- Correctly rejects the question due to insufficient context.
- Or incorrectly generates an answer from noisy documents.

Researchers found that LLMs often fail to reject and still generate an answer.

Teams will evaluate this behavior using open-source LLMs.

16. RGB Ability 3: Information Integration

Information integration checks whether the LLM can combine information from multiple documents to answer a complex question.

Example question:

When were the ChatGPT app for iOS and ChatGPT API launched?

This question has two sub-questions:

1. ChatGPT app for iOS launch date
2. ChatGPT API launch date

The answer to each sub-question may be present in different documents.

The expected answer should include both dates.

Example expected dates mentioned:

- May 18
- March 1

The evaluation checks whether the LLM:

- Integrates information from multiple documents.
 - Gives a complete answer.
 - Avoids answering only one part of the question.
-

17. RGB Ability 4: Counterfactual Robustness

Counterfactual robustness checks whether the LLM can detect and correct factual inconsistencies in the provided context.

Example:

Question asks about the city that hosted the 2004 Olympic Games.

Provided documents incorrectly say:

New York

But from pre-training knowledge, the LLM may know that the correct answer is:

Athens

The expected behavior is for the LLM to identify the inconsistency and respond along the lines of:

There are factual errors in the provided documents. The answer should be Athens.

This ability tests whether the LLM can:

- Detect factual errors in the given context.
 - Correct the answer using its knowledge.
 - Avoid blindly trusting incorrect context.
-

18. RGB Evaluation Metrics

The RGB paper provides evaluation details.

Metrics include:

For Noise Robustness

- Accuracy

For Information Integration

- Accuracy

For Negative Rejection

- Rejection rate

For Counterfactual Robustness

- Error detection rate
- Error correction rate

Teams should refer to the RGB paper for the exact definitions and evaluation methodology.

19. RGB Dataset Clarification

The RGB dataset is annotated.

It contains information such as:

- Positive documents
- Negative documents
- Correct/reference answers

For counterfactual robustness, the dataset contains the correct answer, such as Athens in the Olympic Games example.

The LLM-generated answer should be compared against the reference answer.

20. RGB Experimentation

Teams will experiment with around **4–5 open-source LLMs**.

The objective is to evaluate the four RGB abilities:

- Noise robustness
- Negative rejection
- Information integration
- Counterfactual robustness

RGB is a separate phase from RAGBench.

21. Proposal Document Structure

The proposal document should ideally contain the following sections:

1. Introduction / problem statement
 2. Dataset description
 3. Methodology
 4. Evaluation metrics
 5. Deployment plan and tentative timeline
-

21.1 Introduction / Problem Statement

This section should describe:

- What problem each research paper is trying to solve.
 - What RAGBench addresses.
 - What RGB addresses.
 - Why evaluation and optimization of RAG systems are important.
-

21.2 Dataset Description

Teams should analyze the datasets and describe them clearly.

The dataset section should include exploratory data analysis.

Things to analyze:

- Dataset format
- Domains
- Subdomains
- Number of samples
- Available fields

- Available metrics
 - Type of questions
 - Type of documents/context
 - Any relevant observations from the data
-

21.3 Methodology

This section should explain the approach followed in the research papers and the team's proposed approach.

It should cover:

- How the problem is tackled in the papers.
 - How the team plans to build the RAG pipeline.
 - Which components may be experimented with.
 - How experiments will be structured.
 - How different domains will be handled.
-

21.4 Evaluation Metrics

This section should explain the metrics in detail.

For RAGBench:

- Context relevance
- Context utilization
- Completeness
- Adherence

For RGB:

- Noise robustness
- Negative rejection
- Information integration
- Counterfactual robustness
- Accuracy
- Rejection rate
- Error detection rate
- Error correction rate

Teams should understand the definitions, formulas, and evaluation methodology from the papers.

21.5 Deployment Plan

Teams should describe how they plan to deploy or demonstrate the project.

Possible options:

- Gradio UI
- Streamlit UI
- FastAPI
- Hugging Face Spaces
- Other simple deployment approach

Deployment is not the main focus of the project.

A basic UI is sufficient.

21.6 Tentative Timeline

Teams should include a tentative timeline for project execution.

Example structure:

- First phase: Work on RAGBench.
- Second phase: Work on RGB.
- Final phase: Improve RAGBench/RGB results or attempt bonus components.

Teams can mention how they plan to divide work by:

- Domain
- Pipeline component
- Evaluation metric
- Experiment type

The timeline can be revised later.

The proposal is not fixed forever. Teams are allowed to deviate from it as they learn more.

22. Timeline Discussion

The project duration appears to be around **two months**, likely June and July, with the final presentation possibly planned for **August 9**.

There was some uncertainty about whether the project duration is two months or closer to three months.

Tentative plan discussed:

- RAGBench is experimentally heavier than RGB.
- More time should be spent on RAGBench.
- RGB can be handled after RAGBench.
- Final remaining time can be used for improvements or bonus components.

Possible timeline:

- Around 1 month for RAGBench
- Around 2–3 weeks for RGB
- Remaining time for improvement or bonus work

Earlier rough suggestion:

- First 1 to 1.5 months: RAGBench
- Remaining 1 to 1.5 months: RGB
- Last 2–3 weeks, if available: bonus components or improvements

But if the total project duration is only two months, the timeline should be adjusted accordingly.

23. Team Work Distribution

There are five RAGBench domains and some teams may have four members.

Clarification:

- It is better to cover all five domains.
- Even if there are only four team members, teams can still cover all five domains.
- The underlying RAG principles remain the same across domains.
- Once the skeleton pipeline is ready, teams can change configurations or components for each domain.

Teams may divide domains among members, but should aim to cover all domains.

24. Deployment Clarifications

Deployment is flexible.

Teams can choose:

- Gradio
- Streamlit
- FastAPI
- Hugging Face-based deployment
- Other simple UI

The project is not heavily focused on deployment.

A basic UI is sufficient.

For final presentation, teams may need to demonstrate the system, but the main emphasis is on:

- Evaluation

- Experimentation
 - Analysis
 - Results
-

25. Tools and Restrictions

Teams are not restricted to only the tools mentioned in the papers.

They are allowed to explore better or newer tools.

However:

- They should understand and justify why they are using a particular tool.
 - They should not blindly use tools without analysis.
 - The proposal can mention tentative tools, but teams can change tools later.
-

26. LLM Fine-Tuning

LLM fine-tuning is optional.

It is not expected because most teams will rely on:

- Google Colab
- Kaggle

Fine-tuning LLMs requires GPUs and may not be practical with the available resources.

27. Final Deliverables

Final deliverables will be shared later.

Possibly expected later:

- GitHub repository
- Final presentation
- Project report
- Demo/UI

Exact final deliverables will be communicated as the project progresses.

28. Review Sessions

There will be two review sessions with the professor during the project.

In these sessions:

- Teams will present their progress.
 - The professor will provide feedback.
 - Marks will be allocated for these reviews.
 - These reviews will act as project milestones.
-

29. Bonus Components

Bonus components are optional, but strongly encouraged.

Completing bonus tasks may give:

- Additional marks
 - Better impression in front of professors
 - Stronger final project quality
-

29.1 Bonus Component 1: Improving Chunking and Semantic Similarity

Instead of only using existing chunking strategies such as:

- Fixed-size chunking
- Sliding-window chunking
- Semantic chunking

Teams can explore novel improvements.

Examples:

- Modify existing chunking strategies for a specific domain.
- Train or fine-tune a domain-specific chunking method.
- Create a chunking model using domain-specific data.
- Improve semantic similarity techniques for specific domains.

Example:

For biomedical research, if standard chunking strategies do not work well, a team can attempt to fine-tune or design a biomedical-specific chunking strategy and check whether it improves results.

29.2 Bonus Component 2: Improving RGB Abilities Without Changing the LLM

For RGB, teams can explore whether the LLM's abilities can be improved without changing the base model.

Example:

Suppose Llama 3 8B gets a noise robustness score of 0.7.

Can the team improve it to:

- 0.8
- 0.9
- 1.0

without changing the base model?

Teams can try to improve:

- Noise robustness
- Negative rejection
- Information integration
- Counterfactual robustness

Possible techniques may include better prompting, context formatting, instruction design, or other non-fine-tuning approaches.

29.3 Bonus Component 3: Enterprise RAGBench

Enterprise RAGBench is a recent paper from May.

It is similar to RAGBench but focused on enterprise data.

The dataset includes enterprise-style sources such as:

- Slack messages
- Email conversations
- Fireflies data
- GitHub commits
- Jira
- Confluence

This data is messier than typical benchmark data.

The challenge is:

- How to build an efficient RAG system on enterprise data.
- How to evaluate a RAG system built on messy enterprise datasets.

This is considered a major bonus component.

30. Important Q&A Summary

Q1. Do we need to evaluate which chunking strategy works best for each domain?

Yes.

Teams should evaluate different chunking strategies for each domain and identify which works best.

Q2. Do we need separate Colab notebooks for each domain?

Not required.

Teams can use:

- One notebook for all domains, or
- Separate notebooks for each domain

Both are acceptable.

Q3. Which vector databases should we try?

Suggested starting points:

- Chroma
- FAISS

Teams can also try:

- Milvus

Trying a couple of vector DBs is encouraged.

Q4. Should the final outcome include a matrix of strategies per domain?

Yes.

The final outcome should show which strategies worked best for each domain, along with reasoning and analysis.

Q5. Can we access the original PDFs behind RAGBench?

No.

Teams will work with the provided JSON/text data.

There is no available way to trace back to original PDFs.

Q6. How do we validate counterfactual robustness?

The RGB dataset contains the correct answer.

The LLM response is compared against the reference answer.

For counterfactual robustness, metrics include:

- Error detection rate
 - Error correction rate
-

Q7. Are RAGBench and RGB two separate tasks?

Yes.

RAGBench and RGB are completely separate.

RAGBench focuses on RAG pipeline evaluation.

RGB focuses on LLM generation abilities.

Q8. Do we have to do both RAGBench and RGB?

Yes.

Both are expected.

The plan is to work on RAGBench first and then move to RGB.

Q9. Are there ready-made libraries for RAGBench evaluation metrics?

No ready-made package is expected.

Teams must implement the metrics from the formulas and definitions given in the RAGBench paper.

Q10. Is evaluation the key part of the project?

Yes.

Building RAG is important, but evaluation is the key focus.

Getting the evaluator right is a major objective.

Q11. Are we scoring the LLM only?

No.

For RAGBench, teams are scoring the entire RAG pipeline, not only the LLM.

The pipeline includes:

- Chunking
 - Embedding
 - Retrieval
 - LLM
 - Other components
-

Q12. Do we need five applications for five domains?

Not necessarily five completely separate applications.

But teams should handle all five domains.

They can use a common pipeline with domain-specific configurations.

Q13. Can each team member take one domain?

Yes.

That is acceptable.

But teams should try to cover all five domains.

Q14. Is a basic UI enough?

Yes.

A basic UI is sufficient.

The focus is not on advanced UI.

Q15. Can we deviate from the proposal later?

Yes.

The proposal is tentative.

Teams are free to explore things not mentioned in the proposal.

Q16. Should we stick only to tools from the papers?

No.

Teams can use better or newer tools if they want.

There is no restriction.

Q17. Is LLM fine-tuning required?

No.

LLM fine-tuning is optional and likely impractical due to GPU/resource limitations.

Q18. Will final GitHub repo submission be required?

Final deliverables will be shared later.

A GitHub repository may be expected, but exact requirements will be communicated later.

Q19. Will breakout rooms be created?

Yes.

From next week onward, breakout rooms will be created for each group.

For the current session, Raju may close and restart the session to open breakout rooms so teams can discuss proposals.

Q20. What is the third paper for?

The third paper, described as a “best practices in RAG” paper, is the reference material or textbook for the project.

It explains how to build RAG systems efficiently and how to improve different RAG components.

31. Papers / Reference Materials

Teams should go through three papers/materials:

1. RAGBench paper
2. RGB paper
3. Best practices in RAG paper

The third paper is especially important throughout the project because it explains RAG components and improvement techniques in detail.

32. Session Logistics

- Afternoon session is scheduled and may be covered by Dr. Javi.
 - Some participants did not receive the agenda by email.
 - Agenda was shared in the main WhatsApp group and chat window.
 - Breakout rooms were not initially available, but Raju was expected to restart or reopen the session to create them.
 - Teams can take a break while waiting for breakout rooms.
-

33. Key Takeaways

- Proposal is due by tomorrow.
- Proposal walkthrough is required, but PPT is not required.
- RAGBench and RGB are both required.
- RAGBench comes first and is experimentally heavier.
- RGB comes later and focuses on LLM generation abilities.
- Evaluation is the most important part of the project.
- Teams must implement evaluation metrics, not just use existing libraries.
- Teams should optimize the full RAG pipeline, not just the LLM.
- All five RAGBench domains should ideally be covered.
- Deployment can be simple.
- Bonus components are optional but recommended for extra marks and stronger evaluation.